

MapReduce: Complexity

Baptiste Jeudy

`baptiste.jeudy@univ-st-etienne.fr`

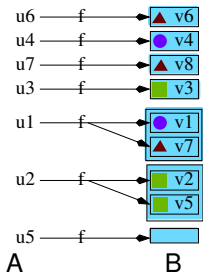
Laboratoire Hubert Curien,
Université Jean-Monnet Saint-Etienne.

Master MLDM DSC

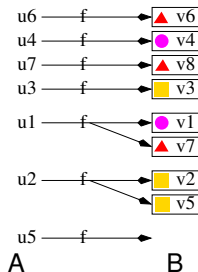
MapReduce Framework : map and flatMap

- Input data: list $A = [u_1, u_2, \dots]$
- Function f applied on each element of A .
 - Result of $A.\text{map}(f)$ is $B = [f(u_1), f(u_2), \dots]$
 - Result of $A.\text{flatMap}(f)$ is $B = f(u_1) + f(u_2) + \dots$
where $+$ is list concatenation

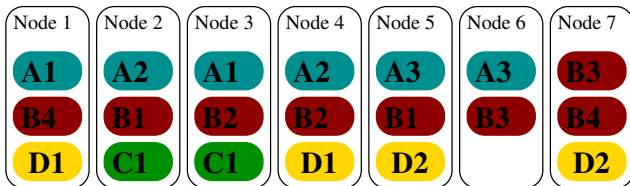
$B=A.\text{map}(f)$



$B=A.\text{flatMap}(f)$



Distribution of Map computations

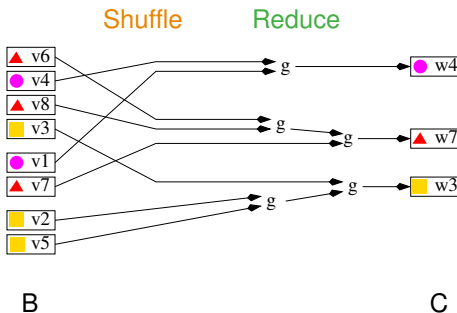


- Assume input data A in file B
- To minimize costly data transfer:
 - Map computed close to the node where the data is (same node or same rack). E.g.,
 - Map for first half of block B1 done on node 2
 - Map for second half of block B1 done on node 5
 - Map for first half of block B2 done on node 3
 - ...
 - The result D' of each map kept in local memory or written on local file
 - result of Map on first half of B1 kept on node 2
 - ...

MapReduce Framework : reduceByKey

- Input data: list $B = [(k1, v1), (k2, v2), \dots]$
- Function g applied to reduce values with the same key.
- One pair (key, result_of_reduction) for each key in B.
 - Result of $B.reduceByKey(g)$ is:

$C = B.reduceByKey(g)$



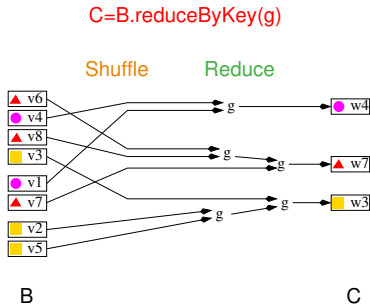
Distribution of Reduce Tasks

- Master node assigns reduce tasks to nodes
 - ie., assigns keys to nodes
- Each Reduce node:
 - **Shuffle:** reads all (key, value) on each map node for each key assigned to it
 - **Slow!: network access**
 - (Need to wait for all previous map/reduce tasks to finish).
 - then compute **Reduce** function on all values with the same key (tree computation) to get one value for each key
 - store the result locally (in memory or in local file)

Optimization of reduceByKey: pre-reduce

- Do some g computations before **Shuffle!**

```
C=B.reduceByKey(g)
```



Handling Hardware Failure

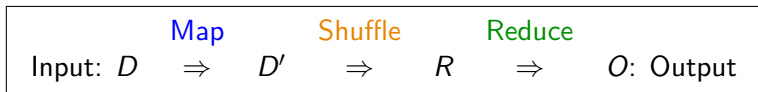
In case of a node failure

- Master node detects the failure
- Master sends the aborted Map / Reduce task to another node
- Map / Reduce Task is run again on the other node

Complexity Analysis

Assume a **Map**(f) followed by a **ReduceByKey**(g)

- Input: D list of objects
- **Map**: For each v in D compute $f(v)$
 $\Rightarrow$ result D' : list of pairs (key, value)
- **ReduceByKey**:
 - **Shuffle**: group pairs of D' by key
 $\Rightarrow$ result R : same as D' (just grouped by key)
 - **Reduce**: compute one pair (key,value) for each group using reduction function g . $\Rightarrow$ result O
- Output: O set of pairs



Complexity Analysis

Map(f)

- Reading input data D distributed on N_m nodes
 - computation close to data: Done on parallel on N_m nodes
 - $\Rightarrow$ local disk read: no network communication
 - $\Rightarrow$ complexity $O(\text{size}(D)/N_m)$
- Compute f for each pair in D
 - Done on parallel on the nodes
 - $\Rightarrow$ complexity $O(\text{size}(D)/N_m)$
 - Generally faster than data read
 - (computation several order of magnitude faster than disk read)

Input: D $\xRightarrow{\text{Map}}$ D' $\xRightarrow{\text{Shuffle}}$ R $\xRightarrow{\text{Reduce}}$ O : Output

Complexity Analysis

Shuffle

Each **Reduce** must get all pairs of D' with the same key

- these pairs are distributed on different nodes
 - $\Rightarrow$ must be sent on network to group by key
 - (R has the same elements as D' in a different order and distributed differently)
- $\Rightarrow$ communication cost: total size of data sent on network
 - this is the size of D' (output of **Map**)
 - Important parameter: replication factor

$$r = \frac{\text{size}(D')}{\text{size}(D)} = \frac{\text{size}(\text{map output})}{\text{size}(\text{input})}$$

- Complexity $O(\text{size}(D')) = O(r \cdot \text{size}(D))$



Complexity Analysis

Reduce

Compute **Reduce** function g for each key of R (compute tree for each key).

- R is D' grouped by key, so $size(R) = size(D') = r.size(D)$
- **Reduce** done in parallel on N_r nodes
 - Data on each node $\approx size(R)/N_r$
 - So complexity on each node $\approx O(size(R)/N_r)$
- Each **Reduce** get at least one key
- Important parameter:
 q = maximum number of elements with the same key
(i.e., maximum size of a compute tree)
- Thus
 - Complexity $> q$
 - Complexity $> O(size(R)/N_r)$
- Also we want q less than a node RAM

$$\text{Complexity} \approx \max(q, size(R)/N_r)$$

Complexity: Total

N_m nodes for **Map** and N_r nodes for **Reduce**.

- **Map:**

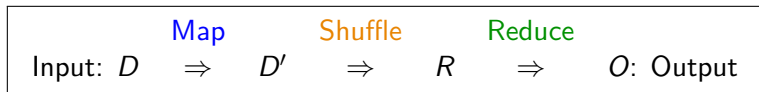
- local disk read: $O(\text{size}(D)/N_m)$
- computation: $O(\text{size}(D)/N_m)$

- **Shuffle:**

- network communication: $O(r.\text{size}(D))$ (r is replication factor)

- **Reduce:**

- computation: $\max(q, O(\text{size}(R)/N_r))$
(q : max number of element with same key)

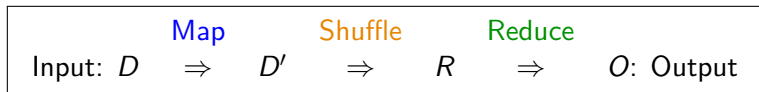


$$\text{size}(R) = \text{size}(D') = r.\text{size}(D)$$

Complexity: Total

N_m nodes for **Map** and N_r nodes for **Reduce**.

- **Map**:
 - local disk read: $O(\text{size}(D)/N_m)$
 - computation: $O(\text{size}(D)/N_m)$
- **Shuffle**: Dominant part in general :
 - network communication: $O(r.\text{size}(D))$ (r is replication factor)
- **Reduce**:
 - computation: $\max(q, O(\text{size}(R)/N_r))$
(q : max number of element with same key)



$$\text{size}(R) = \text{size}(D') = r.\text{size}(D)$$

Example: Matrix Multiplication

- Input: matrices $A = (a_{i,j})$ and $B = (b_{i,j})$
- Compute matrix product $C = A.B$
- Output: $C = (c_{i,j})$ with $c_{i,j} = \sum_k a_{i,k} b_{k,j}$.

Example: Matrix Multiplication

- Input: matrices $A = (a_{i,j})$ and $B = (b_{i,j})$
- Compute matrix product $C = A.B$
- Output: $C = (c_{i,j})$ with $c_{i,j} = \sum_k a_{i,k} b_{k,j}$.

$$\begin{pmatrix} 3 & 2 & 2 \\ 1 & 1 & 0 \\ -1 & 3 & 5 \end{pmatrix} \times \begin{pmatrix} 2 & 2 \\ 1 & 3 \\ -1 & 4 \end{pmatrix} = \begin{pmatrix} 6 & 20 \\ 3 & 5 \\ -4 & 27 \end{pmatrix}$$

Example: Matrix Multiplication

- Input: matrices $A = (a_{i,j})$ and $B = (b_{i,j})$
- Compute matrix product $C = A.B$
- Output: $C = (c_{i,j})$ with $c_{i,j} = \sum_k a_{i,k} b_{k,j}$.

$$\begin{pmatrix} 3 & 2 & 2 \\ 1 & 1 & 0 \\ -1 & 3 & 5 \end{pmatrix} \times \begin{pmatrix} 2 & 2 \\ 1 & 3 \\ -1 & 4 \end{pmatrix} = \begin{pmatrix} 6 & 20 \\ 3 & 5 \\ -4 & 27 \end{pmatrix}$$

$$\begin{pmatrix} 3 & 2 & 2 \\ 1 & 1 & 0 \\ -1 & 3 & 5 \end{pmatrix} \times \begin{pmatrix} 2 & 2 \\ 1 & 3 \\ -1 & 4 \end{pmatrix}$$

Example: Matrix Multiplication

- Input: matrices $A = (a_{i,j})$ and $B = (b_{i,j})$
- Compute matrix product $C = A.B$
- Output: $C = (c_{i,j})$ with $c_{i,j} = \sum_k a_{i,k} b_{k,j}$.

$$\begin{pmatrix} 3 & 2 & 2 \\ 1 & 1 & 0 \\ -1 & 3 & 5 \end{pmatrix} \times \begin{pmatrix} 2 & 2 \\ 1 & 3 \\ -1 & 4 \end{pmatrix} = \begin{pmatrix} 6 & 20 \\ 3 & 5 \\ -4 & 27 \end{pmatrix}$$

$$\begin{pmatrix} 3 & 2 & 2 \\ 1 & 1 & 0 \\ -1 & 3 & 5 \end{pmatrix} \times \begin{pmatrix} 2 & 2 \\ 1 & 3 \\ -1 & 4 \end{pmatrix} = \begin{pmatrix} 6 & 20 \\ 3 & 5 \\ -4 & 27 \end{pmatrix} \quad 5 = (1 \times 2) + (1 \times 3) + (0 \times 4)$$

Example: Matrix Multiplication

- Input: matrices $A = (a_{i,j})$ and $B = (b_{i,j})$
- Compute matrix product $C = A.B$
- Output: $C = (c_{i,j})$ with $c_{i,j} = \sum_k a_{i,k} b_{k,j}$.
- Assume matrices $A = (a_{i,j})$ and $B = (b_{i,j})$ are square ($n \times n$)
 - $\Rightarrow C$ is also $n \times n$.
- Input representation:
 - Matrix A , list of pairs: $[("A", (i, \text{row}_i)) \text{ for } i \leq n]$
 - Matrix B , list of pairs: $[("B", (i, \text{column}_i)) \text{ for } i \leq n]$
 - row_i and column_i are lists of values
 - Size of input: $\text{size}(D) = O(n^2)$.
- Output representation:
 - Matrix C is a list: $[(i, j, c_{i,j}) \text{ for } i, j \leq n]$.

First Solution

- Each reduce computes one output $(i, j, c_{i,j})$ with $c_{i,j} = \sum_k a_{i,k} b_{k,j}$
- Reduce need row i of A and column j of B
- $\Rightarrow$ The key for shuffle is (i, j)
- row_i of A needed for each reduce $(i, .)$
- $column_j$ of B needed for each reduce $(., j)$

Map:

- for each row_i of "A": return n pairs $((i, j), row_i)$ for $j = 1 \dots n$
- for each $column_j$ of "B": return n pairs $((i, j), column_j)$ for $i = 1 \dots n$

$\Rightarrow$ replication factor $r = n$; Size of data sent on network: $O(n^3)$

Reduce

for a key (i, j) : reduce get row_i and $column_j$

$\Rightarrow$ compute $c_{i,j} = \sum_k a_{i,k} b_{k,j}$

2nd Solution

- To decrease the replication factor: each reduce gets several rows/columns:
 - A divided into groups of g rows ($l=1,\dots,n/g$)
 - B divided into groups of g columns ($J=1,\dots,n/g$)
- Each reduce compute all $c_{i,j}$ in a block (l, J) .
- $\Rightarrow$ the key for shuffle is (l, J) .
- Each reduce gets one group of rows l and one group of columns J .
- $\Rightarrow$ each row / column is sent to n/g reducers
- $\Rightarrow$ replication factor $r = n/g$.
- **Size of data sent on network: $O(n^3/g)$**
- if $g = 1$: first solution.
- remark: implementation with just reduceByKey is very clumsy.
(better with spark groupByKey or cogroup reduce-like operations)

2nd Solution

- q : size of data received by a reducer
- $q = 2ng$ (each reducer receive g rows and g columns)
 - So q is at least $2n$
- Size of data sent on network: $2n^3/g = 4n^4/q = O(n^4/q)$
- To reduce complexity: increase q
- But q should be less than memory on a node
- (Pb if $2n$ larger than node memory).

3rd Solution: 1 Map and 2 Reduces

- If we can not increase q (not enough memory or matrices too large):
- Matrices A and B divided into square blocks of size $g \times g$.
- $A(I, J)$ is block at position (I, J) in matrix A . ($1 \leq I, J \leq n/g$)
- Input: list of pairs
 - $[("A", (I, J, A(I, J))) \text{ for } 1 \leq I, J \leq n/g]$
 - $[("B", (I, J, B(I, J))) \text{ for } 1 \leq I, J \leq n/g]$
- Then $C(I, J) = \sum_K C_K(I, J)$ with $C_K(I, J) = A(I, K).B(K, J)$.
- First Step:
 - Map: Split matrices A and B in blocks
 - Reduce: Compute block products $C_K(I, J)$ for all I, J, K .
- Second Step:
 - Reduce: compute $C(I, J)$ for all I, J .

1st Step

Map 1

- If Input = ("A", ($I, K, A(I, K)$))
 - $\Rightarrow$ return $[((I, J, K), ("A", A(I, K))) \text{ for all } 1 \leq J \leq n/g]$.
- If Input = ("B", ($K, J, B(K, J)$))
 - $\Rightarrow$ return $[((I, J, K), ("B", B(K, J))) \text{ for all } 1 \leq I \leq n/g]$.
- Each block of A and B is emitted n/g times
- Shuffle 1: replication factor $r = n/g$.
Size of data sent on network: $O(n^3/g)$.

Reduce 1

- Each reduce gets one block $A(I, K)$ and one block $B(K, J)$
- Compute $C_K(I, J) = A(I, K).B(K, J)$
- return $[((i, j), C_K(i, j)) \text{ for all } (i, j) \text{ in block } (I, J)]$

- Shuffle 2: Size of output of reduce 1 (sent on network)
 - For a given (i, j) key, n/g pairs with this key
 - n^2 different (i, j) pairs.
 - $\Rightarrow$ total size: $O(n^3/g)$

Reduce 2

- Each reduce gets n/g values $C_K(i, j)$ for $1 \leq K \leq n/g$
- Compute and return $C(i, j) = \sum_K C_K(i, j)$

3rd Solution: Complexity

- Data sent during Shuffle:
 - First step: $O(n^3/g)$
 - Second step: $O(n^3/g)$
 - Total: $O(n^3/g)$
- Size of data received by each reduce:
 - First Step: one block (size g^2) of A , and one of B : $q_1 = 2g^2$
 - Second Step: n/g values: $q_2 = n/g$
- Complexity:
 - Take g as large as possible
 - but q_1 and q_2 must be less than a node memory
 - In general, limiting factor is $q_1 = 2g^2$
 - Complexity is then $O(n^3/g) = O(n^3/\sqrt{q})$
 - Generally better than 2nd solution ($O(n^4/q)$)
- Still not possible for very large n (on the order of billions)
 - If matrices have many 0
 - $\Rightarrow$ Use sparse matrix representations.